

AI 기반 개인화 도서 추천 서비스

PROJECT SPECIFICATION

Java · Python · NLP · Recommendation · Spring AI · Deployment

Portfolio Project / Architecture Draft v0.2

2026-07-27

목차

프로젝트의 범위, 기술 구조, 데이터·AI 설계, 배포와 포트폴리오 전략을 순서대로 정리했습니다.

1. 프로젝트 한 줄 정의	6
1.1 확정된 핵심 기술 결정	6
2. 이 프로젝트에서 보여줄 역량	6
3. 핵심 사용자와 사용 시나리오	7
3.1 비회원	7
3.2 회원	7
3.3 관리자	7
4. 개발 범위	7
4.1 MVP: 반드시 완성할 범위	7
4.2 2차 범위	7
4.3 당장 제외할 범위	8
5. 권장 기술 스택	8
5.1 프론트엔드	8
5.2 Java 애플리케이션	9
5.3 Python AI 애플리케이션	9
5.4 데이터와 인프라	10
6. 전체 아키텍처	10
6.1 서비스 분리 원칙	11
Spring Boot가 담당하는 것	11
Python이 담당하는 것	11
6.2 Spring Boot와 FastAPI의 통신 원칙	11
7. 데이터 확보 전략	11
7.1 도서 기본 정보	11
7.2 리뷰 데이터	12
7.3 리뷰를 확보하지 못했을 때의 대안	12
8. 기능 요구사항	13
8.1 인증	13

8.2 도서	13
8.3 사용자 행동	13
8.4 리뷰 분석	13
8.5 추천	14
8.6 트렌드	14
8.7 챗봇	14
9. 주요 화면	14
10. 데이터베이스 초안	15
10.1 주요 인덱스	17
11. REST API 초안	17
인증	17
도서	17
사용자 행동과 취향	17
추천	18
챗봇	18
내부 Python API	18
11.1 FastAPI 상세 설계	19
FastAPI의 책임	19
FastAPI가 담당하지 않는 것	19
감성 분석 요청 예시	19
실행과 모델 로딩	20
Spring Boot 장애 대응 정책	20
관측 지표	20
12. 감성 분석 설계	20
12.1 문제 정의	20
12.2 모델 비교	21
12.3 데이터 라벨링	21
12.4 데이터 분리	21
12.5 평가 지표	22
12.6 도서 감성 점수	22
13. 추천 시스템 설계	22
13.1 1단계: 콘텐츠 기반 추천	22
13.2 2단계: 행동 점수	22

13.3 3단계: 트렌드 점수	23
13.4 하이브리드 랭킹	23
13.5 다양성	23
13.6 콜드 스타트	23
13.7 추천 평가	24
14. Spring AI 챗봇 설계	24
14.1 챗봇 역할	24
14.2 제공할 Tool	24
14.3 처리 흐름	24
14.4 환각 방지	25
15. 배치와 모델 운영	25
15.1 정기 작업	25
15.2 작업 상태	25
15.3 모델 버전	26
16. 보안과 개인정보	26
17. 비기능 요구사항	27
성능 목표	27
안정성	27
데이터 품질	27
18. 테스트 전략	27
Java	27
Python	27
E2E	28
성능	28
19. 배포 구조	28
포트폴리오용 권장안	28
CI/CD	29
20. 저장소 구조	29
21. 개발 일정 예시: 12주	30
22. 포트폴리오 문서에 반드시 넣을 자료	30
README 첫 화면	30

AI 모델 문서	31
추천 시스템 문서	31
백엔드 문서	31
기술 의사결정 기록 예시	31
23. 면접에서 설명할 핵심 이야기	32
24. 완료 기준	32
25. 지금 결정해야 할 사항	33
26. 참고한 공식 자료	33

문서 상태: 기술 구성 확정 초안 v0.2

작성 기준일: 2026-07-27

프로젝트 목적: 취업 포트폴리오용 Java·Python·DB·AI·배포 통합 프로젝트

1. 프로젝트 한 줄 정의

도서와 리뷰 데이터를 수집·분석하고, 사용자의 검색·조회·찜·좋아요 행동을 바탕으로 개인화 도서를 추천하며, Spring AI 챗봇을 통해 자연어로 검색과 추천을 제공하는 웹 서비스.

임시 프로젝트명은 BookPick AI로 사용한다. 실제 개발을 시작할 때 서비스명과 저장소명은 변경해도 된다.

1.1 확정된 핵심 기술 결정

영역	확정 기술	역할
사용자 웹	React + TypeScript	검색, 도서 상세, 추천 목록, 차트, 챗봇 스트리밍 UI
공개 백엔드	Java 21 + Spring Boot	인증·인가, 도메인 로직, REST API, 사용자 행동, 챗봇 오케스트레이션
AI 서빙	Python 3.12 + FastAPI	감성 분석, 임베딩, 추천 후보 생성 등 실시간 모델 추론
데이터·학습	Python Batch + PyTorch	데이터 수집·정제, 모델 학습·평가, 일괄 분석
데이터베이스	PostgreSQL + pgvector	서비스 데이터, 행동 로그, 임베딩과 유사도 검색

React와 Thymeleaf를 중복 사용하지 않는다. 사용자 화면은 React로 통일하고 Spring Boot는 JSON REST API를 제공한다. FastAPI는 외부 사용자에게 공개하지 않는 AI 전용 내부 서비스로 운영한다.

2. 이 프로젝트에서 보여줄 역량

분야	보여줄 내용
Java 백엔드	Spring Boot, REST API, 인증·인가, JPA, 트랜잭션, 예외 처리, 테스트
Python	데이터 수집, 정제, NLP 모델 학습·평가, 추천 모델, 추론 API
AI/ML	베이스라인과 RNN 계열 모델 비교, 감성 분석, 임베딩, 추천 알고리즘
생성형 AI	Spring AI, Tool Calling, RAG 또는 검색 결과 기반 응답
데이터베이스	관계형 모델링, 인덱스, PostgreSQL, pgvector, 행동 로그
인프라	Docker, CI/CD, HTTPS, 운영 로그, 모니터링, 클라우드 배포
협업 역량	명세, ERD, API 문서, 이슈 관리, 테스트 전략, 기술 의사결정 기록

중요한 목표는 기술을 많이 나열하는 것이 아니라 다음 흐름을 완성하는 것이다.

TEXT

데이터 확보 → 정제 → 모델 학습 → 성능 평가 → 서비스 연동

→ 사용자 행동 수집 → 개인화 추천 → 배포 → 운영 지표 확인

3. 핵심 사용자와 사용 시나리오

3.1 비회원

- 제목, 작가, ISBN으로 책을 검색한다.
- 인기 급상승 도서와 리뷰 분석 결과를 본다.
- 회원가입 후 사용할 수 있는 개인화 기능을 확인한다.

3.2 회원

- 선호 장르와 좋아하는 책을 선택해 초기 취향을 등록한다.
- 책을 검색하고 상세 정보와 리뷰 분석 결과를 본다.
- 책을 찜하거나 좋아요·싫어요로 평가한다.
- 자신의 행동을 기반으로 한 추천 목록과 추천 이유를 확인한다.
- 챗봇에게 자연어로 책을 검색하거나 추천받는다.

3.3 관리자

- 데이터 수집 작업의 성공·실패 상태를 확인한다.
- 감성 분석 및 추천 모델 버전을 확인한다.
- 비정상적인 리뷰 데이터나 중복 도서를 관리한다.

4. 개발 범위

4.1 MVP: 반드시 완성할 범위

1. 회원가입, 로그인, 로그아웃
2. 도서 검색 및 상세 조회
3. 장르 선택, 찜, 좋아요·싫어요
4. 리뷰 감성 분석 결과 제공
5. 사용자별 콘텐츠 기반 추천
6. 최근 행동 기반 인기·트렌드 도서
7. Spring AI 챗봇의 도서 검색 및 추천
8. Docker 기반 운영 환경 배포
9. 핵심 API 테스트와 모델 평가 리포트

4.2 2차 범위

- 협업 필터링 또는 행렬 분해 기반 추천

- 추천 결과에 대한 피드백 수집
- 리뷰의 장점·단점 및 항목별 감성 분석
- 이메일 인증 또는 소셜 로그인
- 관리자 대시보드
- 검색 자동완성 및 오타 보정

4.3 당장 제외할 범위

- 모바일 앱
- 실시간 채팅 상담
- 결제와 도서 판매
- Kafka 기반 대규모 이벤트 처리
- Kubernetes
- 마이크로서비스 수십 개로 분리
- 직접 대규모 언어 모델을 사전학습하는 작업

제외 항목은 기술적으로 못해서가 아니라, 완성도와 개발 기간을 지키기 위한 선택이다.

5. 권장 기술 스택

5.1 프론트엔드

기술	용도	선택 이유
React + TypeScript	웹 UI	취업 포트폴리오와 컴포넌트 기반 개발에 적합
Vite	빌드 도구	설정이 단순하고 개발 속도가 빠름
React Router	화면 라우팅	검색·상세·마이페이지 경로 구성
TanStack Query	서버 상태	캐시, 재시도, 로딩·에러 상태 관리
Zustand 또는 Context	클라이언트 상태	인증 표시와 간단한 UI 상태
Tailwind CSS 또는 CSS Modules	스타일	둘 중 하나만 선택
Vitest + React Testing Library	테스트	핵심 UI 단위 테스트

5.2 Java 애플리케이션

기술	용도
Java 21 LTS	백엔드 언어
Spring Boot 4.0.x	애플리케이션 프레임워크
Spring Web MVC	REST API
Spring Data JPA	영속성
Spring Security	인증·인가
Bean Validation	요청값 검증
Spring AI 2.0.x	챗봇, Tool Calling, 필요 시 RAG
Flyway	DB 마이그레이션
springdoc-openapi	Swagger API 문서
JUnit 5, Mockito, Testcontainers	단위·통합 테스트
Gradle	빌드

Spring AI 2.0.x는 Spring Boot 4.0.x 계열과 맞춰 사용한다. 개발 시작 시 공식 호환표를 다시 확인하고 Snapshot이 아닌 정식 릴리스를 고정한다.

5.3 Python AI 애플리케이션

기술	용도
Python 3.12	데이터·AI 서비스
FastAPI	내부 추론 API
Uvicorn	FastAPI ASGI 서버
Pydantic	요청·응답 스키마
SQLAlchemy 또는 psycopg	DB 접근
pandas, NumPy	데이터 처리
scikit-learn	베이스라인·평가·추천
PyTorch	LSTM/BiLSTM 모델
Transformers	한국어 사전학습 모델 비교
MLflow	선택 사항: 실험·모델 버전 관리
pytest	테스트
ruff	린트와 포맷 검사

5.4 데이터와 인프라

기술	용도
PostgreSQL	서비스 데이터와 행동 로그
pgvector	도서 설명·키워드 임베딩 저장
Redis	선택 사항: 캐시, 요청 제한
Docker Compose	로컬·운영 컨테이너 구성
Nginx 또는 Caddy	HTTPS와 리버스 프록시
GitHub Actions	테스트·빌드·배포 자동화
Prometheus + Grafana	2차 범위: 메트릭
Sentry 또는 OpenTelemetry	2차 범위: 오류 추적

MVP에서는 PostgreSQL 하나로 시작한다. 검색 전용 Elasticsearch, 메시지 브로커, Kubernetes는 실제 병목이 생길 때만 도입한다.

6. 전체 아키텍처

DIAGRAM DEFINITION

flowchart TB

USER["사용자 브라우저"]

WEB["React Web"]

JAVA["Spring Boot API"]

PYAPI["Python FastAPI"]

BATCH["Python Data/ML Batch"]

DB["PostgreSQL + pgvector"]

CACHE["Redis - 선택"]

BOOKAPI["공식 도서 API"]

REVIEW["하용된 리뷰 데이터 소스"]

LLM["LLM Provider 또는 로컬 모델"]

USER --> WEB

WEB --> JAVA

JAVA --> DB

JAVA -. 캐시 .-> CACHE

JAVA -->|"감성·추천 추천"| PYAPI

JAVA -->|"Spring AI"| LLM

LLM -->|"Tool 요청"| JAVA

BATCH --> BOOKAPI

BATCH --> REVIEW

BATCH -->|"정제 데이터·모델 결과"| DB

PYAPI --> DB

6.1 서비스 분리 원칙

Spring Boot가 담당하는 것

- 외부에 공개되는 단일 API 진입점
- 인증 및 사용자 권한
- 도서 검색·조회
- 사용자 행동 기록
- 찜과 평가
- 추천 결과 조합과 응답
- 챗봇 대화와 도구 실행
- 관리자 API

Python이 담당하는 것

- 도서·리뷰 수집
- 텍스트 정제와 중복 제거
- 감성 분석 모델 학습·평가
- 도서 임베딩 생성
- 추천 후보 생성 또는 모델 추론
- 정기적인 모델 산출물 업데이트

브라우저가 Python API를 직접 호출하지 않는다. 모든 외부 요청은 Spring Boot를 통하게 해 인증과 응답 형식을 일관되게 유지한다.

6.2 Spring Boot와 FastAPI의 통신 원칙

- React는 FastAPI 주소를 알지 못하며 Spring Boot만 호출한다.
- Spring Boot는 인증된 사용자와 도서 정보를 검증한 후 필요한 데이터만 FastAPI에 전달한다.
- FastAPI 응답에는 `modelVersion`, `inferenceTimeMs`, `requestId`를 포함한다.
- 두 서비스는 OpenAPI 스키마와 버전이 있는 DTO를 사용한다.
- 호출에는 `connect timeout`과 `read timeout`을 설정한다.
- FastAPI 장애 시 Spring Boot는 저장된 이전 추천·감성 결과를 반환한다.
- 동일한 `traceId`를 전달해 Spring Boot와 FastAPI 로그를 연결한다.
- FastAPI 내부 API는 Docker 내부 네트워크에서만 접근 가능하게 한다.

7. 데이터 확보 전략

7.1 도서 기본 정보

1순위는 정식 Open API다.

- 국립중앙도서관 ISBN 서지정보 API
- ISBN, 제목, 저자, 출판사, 발행일 등 기본 서지정보 확보

- 도서관 정보나루 등 사용 승인을 받은 공공 API
- Open Library
- 해외 도서나 표지 보완이 필요할 때 검토
- 알라딘 TTB
- 약관, 출처 표시, 링크 제공 조건과 허용 필드를 확인한 뒤 보조 소스로 검토

네이버 책 검색 API는 2026-07-31 종료 예정이므로 신규 프로젝트의 핵심 데이터 소스로 사용하지 않는다.

7.2 리뷰 데이터

리뷰 데이터가 프로젝트의 가장 큰 위험 요소다. 사이트 화면에 공개되어 있다는 사실이 대량 수집과 재사용을 허용한다는 의미는 아니다.

수집 전에 다음을 기록한다.

- 사이트 이용약관
- robots.txt
- API 제공 여부
- 자동 수집 허용 여부
- 저장·가공·재배포 허용 범위
- 호출 주기와 요청량 제한
- 출처 표시 의무
- 리뷰 원문 보관 가능 여부

안전한 기본 정책:

- 가능하면 리뷰 원문 전체를 외부에 다시 노출하지 않는다.
- 원문 대신 리뷰 ID, 출처 URL, 수집 시각, 별점, 파생 감성 점수와 키워드를 저장한다.
- 삭제 요청이나 원문 삭제를 반영할 수 있도록 출처 식별자를 유지한다.
- User-Agent, 요청 간격, 재시도 횟수와 백오프를 명시한다.
- 수집 불가 사이트를 우회하지 않는다.

7.3 리뷰를 확보하지 못했을 때의 대안

- 서비스 내부에서 사용자가 직접 리뷰를 작성하도록 한다.
- 사용 허가가 명확한 공개 데이터셋을 학습·검증에 사용한다.
- 소규모 리뷰를 직접 라벨링해 모델 비교 실험을 진행한다.
- 도서 소개와 메타데이터 기반 추천을 MVP로 먼저 완성한다.

이 대안을 마련해 두면 외부 사이트 정책 변경 때문에 프로젝트 전체가 중단되지 않는다.

8. 기능 요구사항

8.1 인증

ID	요구사항
AUTH-01	사용자는 이메일과 비밀번호로 가입할 수 있다.
AUTH-02	비밀번호는 단방향 해시로 저장한다.
AUTH-03	로그인 성공 시 Access Token과 Refresh Token을 발급한다.
AUTH-04	Refresh Token 재사용과 탈취 대응 정책을 적용한다.
AUTH-05	회원 탈퇴 시 개인정보와 행동 데이터 처리 정책을 안내한다.

8.2 도서

ID	요구사항
BOOK-01	제목, 저자, ISBN으로 검색할 수 있다.
BOOK-02	장르, 출간 연도, 감성 점수로 필터링할 수 있다.
BOOK-03	상세 화면에 책 정보, 리뷰 통계, 키워드, 추천 근거를 표시한다.
BOOK-04	같은 ISBN의 중복 도서를 하나로 정규화한다.
BOOK-05	데이터 출처와 마지막 갱신 시각을 표시한다.

8.3 사용자 행동

ID	요구사항
ACT-01	검색, 상세 조회, 찜, 좋아요, 싫어요 이벤트를 기록한다.
ACT-02	동일 요청의 중복 기록을 줄이기 위한 event_id를 사용한다.
ACT-03	사용자는 찜과 평가를 취소할 수 있다.
ACT-04	사용자는 개인화 추천에 사용되는 데이터 종류를 확인할 수 있다.

8.4 리뷰 분석

ID	요구사항
NLP-01	리뷰별 긍정·중립·부정 확률을 저장한다.
NLP-02	도서별 감성 점수와 신뢰도를 집계한다.
NLP-03	분석 결과에 모델 버전과 분석 시각을 저장한다.
NLP-04	리뷰 수가 적으면 낮은 신뢰도임을 UI에 표시한다.
NLP-05	2차 범위에서 스토리·문체·번역·가독성 등 항목별 감성을 제공한다.

8.5 추천

ID	요구사항
REC-01	신규 회원에게 온보딩 취향 기반 추천을 제공한다.
REC-02	기존 회원에게 행동 기반 개인화 추천을 제공한다.
REC-03	각 추천에는 최소 한 개의 추천 이유를 표시한다.
REC-04	이미 싫어요한 책은 기본 추천에서 제외한다.
REC-05	동일 작가·장르만 과도하게 노출되지 않도록 다양성을 적용한다.
REC-06	추천 결과의 모델 버전과 생성 시각을 기록한다.

8.6 트렌드

ID	요구사항
TREND-01	최근 기간의 검색·조회·찜·좋아요를 가중 합산한다.
TREND-02	시간 감쇠를 적용해 오래된 이벤트의 영향력을 낮춘다.
TREND-03	전체·장르별 인기 급상승 목록을 제공한다.
TREND-04	최소 이벤트 수 미만의 결과는 순위에서 제외해 노이즈를 줄인다.

8.7 챗봇

ID	요구사항
CHAT-01	제목·작가·장르 기반 자연어 검색을 지원한다.
CHAT-02	로그인 사용자의 취향을 반영한 추천을 제공한다.
CHAT-03	실제 DB 조회 결과만 도서 사실 정보로 사용한다.
CHAT-04	답변에 추천 이유와 상세 페이지 링크를 포함한다.
CHAT-05	결과가 없으면 책을 지어내지 않고 검색 조건 변경을 제안한다.
CHAT-06	프롬프트 입력과 Tool 호출에 사용자 권한 검사를 적용한다.

9. 주요 화면

- 랜딩·홈
 - 오늘의 추천
 - 인기 급상승
 - 장르별 추천
- 회원가입·로그인
- 취향 온보딩

- 선호 장르
 - 좋아하는 책 3권 이상
4. 통합 검색
 5. 도서 상세
 - 서지정보
 - 감성 분포
 - 장점·단점 키워드
 - 유사 도서
 6. 개인 추천
 7. 찜 목록
 8. 활동 기록 및 취향 관리
 9. 챗봇 패널
 10. 관리자 작업 현황

10. 데이터베이스 초안

DIAGRAM DEFINITION

```
erDiagram
    USERS ||--o{ USER_INTERACTIONS : creates
    USERS ||--o{ USER_PREFERENCES : has
    USERS ||--o{ RECOMMENDATIONS : receives
    USERS ||--o{ CHAT_SESSIONS : owns
    CHAT_SESSIONS ||--o{ CHAT_MESSAGES : contains

    BOOKS ||--o{ USER_INTERACTIONS : receives
    BOOKS ||--o{ REVIEWS : has
    BOOKS ||--o{ BOOK_AUTHORS : maps
    AUTHORS ||--o{ BOOK_AUTHORS : writes
    BOOKS ||--o{ BOOK_CATEGORIES : maps
    CATEGORIES ||--o{ BOOK_CATEGORIES : classifies
    REVIEWS ||--o{ REVIEW_ANALYSES : analyzed_as
    BOOKS ||--o{ RECOMMENDATIONS : recommended
    MODEL_VERSIONS ||--o{ REVIEW_ANALYSES : produces
    MODEL_VERSIONS ||--o{ RECOMMENDATIONS : produces

    USERS {
        bigint id PK
        varchar email UK
        varchar password_hash
        varchar nickname
        varchar role
        timestampz created_at
    }

    BOOKS {
        bigint id PK
        varchar isbn13 UK
        varchar title
```

```
text description
varchar publisher
date published_date
varchar cover_url
varchar source
timestampz source_updated_at
vector embedding
}
```

```
REVIEWS {
  bigint id PK
  bigint book_id FK
  varchar source
  varchar source_review_id
  smallint rating
  text normalized_text
  varchar content_hash
  timestampz written_at
  timestampz collected_at
}
```

```
REVIEW_ANALYSES {
  bigint review_id PK,FK
  bigint model_version_id FK
  decimal positive_prob
  decimal neutral_prob
  decimal negative_prob
  jsonb keywords
  timestampz analyzed_at
}
```

```
USER_INTERACTIONS {
  bigint id PK
  uuid event_id UK
  bigint user_id FK
  bigint book_id FK
  varchar event_type
  decimal weight
  jsonb context
  timestampz created_at
}
```

```
RECOMMENDATIONS {
  bigint id PK
  bigint user_id FK
  bigint book_id FK
  bigint model_version_id FK
  decimal score
  jsonb reasons
  timestampz generated_at
}
```

추가 테이블:

- [refresh_tokens](#)
- [user_preferences](#)

- authors, book_authors
- categories, book_categories
- model_versions
- crawl_jobs, crawl_job_errors
- chat_sessions, chat_messages

10.1 주요 인덱스

- books(isbn13) unique
- lower(books.title) 검색 인덱스
- reviews(book_id, written_at desc)
- reviews(source, source_review_id) unique
- user_interactions(user_id, created_at desc)
- user_interactions(book_id, event_type, created_at desc)
- recommendations(user_id, generated_at desc)
- books.embedding pgvector ANN 인덱스

한국어 제목 검색 품질이 부족하면 PostgreSQL trigram 또는 별도의 검색 엔진을 2차로 검토한다.

11. REST API 초안

인증

HTTP

```
POST /api/v1/auth/signup
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
DELETE /api/v1/users/me
```

도서

HTTP

```
GET /api/v1/books?q=&author=&category=&page=&size=
GET /api/v1/books/{bookId}
GET /api/v1/books/{bookId}/review-summary
GET /api/v1/books/{bookId}/similar
GET /api/v1/books/trending?period=7d&category=
```

사용자 행동과 취향

HTTP

```
POST /api/v1/users/me/preferences
GET /api/v1/users/me/preferences
POST /api/v1/books/{bookId}/interactions
```

```
DELETE /api/v1/books/{bookId}/interactions/{type}
GET    /api/v1/users/me/wishlist
```

행동 요청 예시:

```
JSON
{
  "eventId": "85ea24de-7b5c-4de3-b872-f1a65d2855ae",
  "type": "WISHLIST",
  "context": {
    "source": "SEARCH_RESULT",
    "query": "한국 추리 소설"
  }
}
```

추천

```
HTTP
GET  /api/v1/recommendations/me?size=20
POST /api/v1/recommendations/me/refresh
POST /api/v1/recommendations/{recommendationId}/feedback
```

응답 예시:

```
JSON
{
  "bookId": 1024,
  "score": 0.873,
  "reasons": [
    "최근 추리 장르를 자주 조회했습니다.",
    "찜한 도서와 핵심 키워드가 유사합니다.",
    "리뷰에서 몰입감에 대한 긍정 평가가 높습니다."
  ],
  "modelVersion": "hybrid-recommender-0.3.0"
}
```

챗봇

```
HTTP
POST /api/v1/chat/sessions
POST /api/v1/chat/sessions/{sessionId}/messages
GET  /api/v1/chat/sessions/{sessionId}/messages
```

운영에서는 챗봇 응답을 SSE 스트리밍으로 제공할 수 있다.

내부 Python API

외부에 공개하지 않고 내부 네트워크에서만 호출한다.

```
HTTP
GET  /internal/health
GET  /internal/ready
POST /internal/v1/sentiment/predict
POST /internal/v1/sentiment/batch-predict
```

```
POST /internal/v1/embeddings/books
POST /internal/v1/recommendations/candidates
GET /internal/v1/models
```

Python API 장에 시 Spring Boot는 전체 요청을 실패시키기보다 이전 추천 결과나 저장된 감성 결과를 반환하도록 설계한다.

11.1 FastAPI 상세 설계

FastAPI의 책임

- 학습 완료된 감성 분석 모델을 메모리에 적재하고 추론한다.
- 도서 소개와 리뷰 키워드의 임베딩을 생성한다.
- 사용자 취향 벡터를 받아 추천 후보와 점수를 계산한다.
- 현재 적재된 모델 이름, 버전, 로딩 상태를 제공한다.
- 요청별 추론 시간과 모델 처리 지표를 기록한다.

FastAPI가 담당하지 않는 것

- 회원가입, 로그인, JWT 발급
- 사용자 권한과 소유권 판단
- 찜·좋아요·채팅 세션 관리
- 브라우저에 공개되는 범용 REST API
- 모델 학습과 장시간 데이터 수집

모델 학습·크롤링·대량 리뷰 분석은 HTTP 요청 처리 과정에서 실행하지 않고 별도의 Python 배치 명령으로 분리한다.

감성 분석 요청 예시

```
JSON
{
  "requestId": "9d790dd9-3022-4b70-9854-005014aba954",
  "texts": [
    "초반은 느리지만 후반부 몰입감이 매우 좋았습니다."
  ]
}
```

응답 예시:

```
JSON
{
  "requestId": "9d790dd9-3022-4b70-9854-005014aba954",
  "predictions": [
    {
      "label": "POSITIVE",
      "positiveProbability": 0.87,
      "neutralProbability": 0.08,
      "negativeProbability": 0.05
    }
  ],
  "modelVersion": "sentiment-transformer-0.3.0",
  "inferenceTimeMs": 41
}
```

}

실행과 모델 로딩

- FastAPI 애플리케이션 시작 시 운영 모델을 한 번 로딩한다.
- `/internal/health`는 프로세스 생존 여부를 반환한다.
- `/internal/ready`는 모델 로딩과 DB 연결이 완료된 경우에만 성공한다.
- 초기에는 Uvicorn worker 1개로 시작한다.
- worker를 늘리면 모델이 프로세스마다 중복 적재될 수 있으므로 메모리를 먼저 측정한다.
- GPU를 사용할 경우 기본적으로 모델 하나당 worker 1개로 시작한다.
- 반복 요청은 가능한 범위에서 batch inference로 묶는다.

Spring Boot 장애 대응 정책

상황	Spring Boot 처리
FastAPI 정상	최신 추론 결과 사용
Timeout	저장된 이전 결과 반환
일시적 연결 실패	짧은 횟수로 재시도
반복 실패	Circuit Breaker를 열고 즉시 fallback
잘못된 응답 스키마	응답 폐기, 오류 기록, 이전 결과 반환
모델 준비 중	HTTP 503을 받고 이전 결과 반환

재시도는 감성 조회처럼 멍등성이 보장되는 요청에만 적용한다. Timeout, Retry, Circuit Breaker는 Spring Boot 측에서 관리하고 FastAPI 오류가 사용자 API 전체 장애로 번지지 않도록 한다.

관측 지표

- 요청 수와 오류율
- p50, p95, p99 응답 시간
- 모델별 추론 시간
- batch size
- 모델 로딩 시간
- 프로세스 메모리
- CPU 또는 GPU 사용량
- Spring Boot에서 FastAPI까지 이어지는 traceId

12. 감성 분석 설계

12.1 문제 정의

입력:

TEXT

"초반 전개는 느리지만 후반부 몰입감과 결말이 좋았다."

1차 출력:

JSON

```
{
  "positive": 0.72,
  "neutral": 0.09,
  "negative": 0.19,
  "label": "POSITIVE"
}
```

2차 출력:

JSON

```
{
  "aspects": {
    "pace": "NEGATIVE",
    "immersion": "POSITIVE",
    "ending": "POSITIVE"
  }
}
```

12.2 모델 비교

모델	목적
TF-IDF + Logistic Regression	빠르고 설명 가능한 베이스라인
LSTM 또는 BiLSTM	RNN 계열을 직접 구현하고 학습 과정을 보여주기 위함
한국어 사전학습 Transformer	실제 서비스 성능 비교

RNN을 반드시 사용하고 싶다면 최종 모델로 고정하지 말고 실험 모델 중 하나로 둔다. 모델 선택은 테스트 데이터 F1과 추론 비용을 기준으로 결정한다.

12.3 데이터 라벨링

- 별점 4~5: 긍정 후보
- 별점 3: 중립 후보
- 별점 1~2: 부정 후보
- 별점과 본문 감성이 충돌하는 샘플은 수동 검수
- 일정 비율은 두 명 이상이 독립적으로 라벨링
- 동일 문장과 거의 동일한 문장을 중복 제거

별점을 정답으로 그대로 사용하면 노이즈가 생기므로 이를 **weak label**로 명시한다.

12.4 데이터 분리

무작위 리뷰 분리만 사용하면 같은 책의 유사 리뷰가 학습·평가 세트에 동시에 들어갈 수 있다. 가능하면 책 ISBN 단위로 train/validation/test를 분리해 데이터 누수를 줄인다.

12.5 평가 지표

- Macro F1: 주 지표
- 클래스별 Precision, Recall
- Confusion Matrix
- 모델 크기
- 리뷰 한 건당 추론 시간
- CPU 환경 처리량

12.6 도서 감성 점수

예시:

TEXT

```
review_sentiment = positive_probability - negative_probability
weighted_score =  $\Sigma(\text{review\_sentiment} \times \text{reliability} \times \text{recency\_weight}) / \Sigma(\text{weight})$ 
display_score = (weighted_score + 1) \times 50
```

리뷰가 적은 책이 극단적인 점수를 받지 않도록 전체 평균을 이용한 Bayesian smoothing을 적용한다.

13. 추천 시스템 설계

13.1 1단계: 콘텐츠 기반 추천

도서 표현에 다음 특성을 사용한다.

- 장르
- 저자
- 책 소개 임베딩
- 리뷰 키워드
- 리뷰 감성 점수
- 출간 연도

사용자가 좋아요·찜한 도서 벡터의 가중 평균을 사용자 취향 벡터로 만들고, cosine similarity가 높은 책을 후보로 생성한다.

13.2 2단계: 행동 점수

초기 가중치 예시:

이벤트	가중치
검색 결과 노출	0.1
상세 조회	1.0
반복 상세 조회	1.5
좋아요	3.0

이벤트	가중치
찜	4.0
싫어요	-5.0

이 값은 정답이 아니라 초기 가설이다. 이후 사용자 테스트 결과로 조정한다.

13.3 3단계: 트렌드 점수

TEXT

```
event_score =
view × 1
+ search_click × 1.5
+ like × 3
+ wishlist × 4

decayed_score = event_score × exp(-λ × elapsed_days)
```

단순 누적 조회 수 대신 시간 감쇠를 사용해 “요즘 많이 찾는 책”을 표현한다.

13.4 하이브리드 랭킹

초기 예시:

TEXT

```
final_score =
0.50 × content_similarity
+ 0.20 × collaborative_score
+ 0.15 × trend_score
+ 0.10 × sentiment_score
+ 0.05 × freshness_score
```

협업 필터링 데이터가 부족한 MVP에서는 해당 비중을 콘텐츠·트렌드 점수에 나눠 준다.

13.5 다양성

상위 점수만 그대로 노출하면 같은 작가와 장르가 반복될 수 있다.

- 같은 작가는 한 화면에 최대 2권
- 한 장르가 전체의 60%를 넘지 않도록 제한
- MMR 방식으로 관련성과 다양성 균형
- 이미 조회했거나 싫어요한 책에 패널티

13.6 콜드 스타트

신규 사용자:

- 선호 장르 3개 선택
- 좋아하는 책 3권 이상 선택
- 길이, 분위기, 난이도 선택
- 전체 트렌드와 취향 벡터 결합

신규 도서:

- 저자, 장르, 소개 임베딩만으로 후보에 포함
- 행동 데이터가 쌓이면 점진적으로 개인화 점수 반영

13.7 추천 평가

오프라인:

- Precision@K
- Recall@K
- NDCG@K
- Coverage
- Diversity

온라인 또는 사용자 테스트:

- 추천 클릭률
- 상세 조회 전환율
- 찜 전환율
- “추천이 도움이 되었나요?” 긍정 비율

실제 사용자가 적을 때는 오프라인 지표와 5~10명의 사용성 테스트를 분리해 보고한다. 합성 데이터를 실제 사용자 성과처럼 표현하지 않는다.

14. Spring AI 챗봇 설계

14.1 챗봇 역할

챗봇은 지식을 기억해 책을 임의로 추천하는 역할이 아니라, 사용자의 자연어 요청을 해석하고 검증된 서비스 기능을 호출하는 인터페이스다.

14.2 제공할 Tool

TEXT

```
searchBooks(query, author, category, limit)
getBookDetail(bookId)
getReviewSummary(bookId)
getTrendingBooks(category, period, limit)
getRecommendationsForCurrentUser(limit)
```

14.3 처리 흐름

DIAGRAM DEFINITION

```
sequenceDiagram
    actor U as 사용자
    participant S as Spring Boot
    participant AI as Spring AI/LLM
```



```

participant T as Book Tools
participant DB as PostgreSQL

U->>S: "내 취향에 맞는 짧은 추리소설 추천해줘"
S->>AI: 사용자 요청 + 허용 Tool 목록
AI->>T: getRecommendationsForCurrentUser()
T->>DB: 사용자 취향·추천 후보 조회
DB-->>T: 검증된 도서 결과
T-->>AI: 도서·점수·추천 이유
AI-->>S: 자연어 답변과 도서 ID
S-->>U: 추천 이유 + 상세 페이지 링크

```

14.4 환각 방지

- DB 검색 결과에 없는 ISBN·책 제목은 사실 정보로 답변하지 않는다.
- 도서 정보에는 내부 `bookId`와 출처를 함께 전달한다.
- Tool 결과가 없으면 "찾지 못했다"고 답한다.
- 사용자 ID를 프롬프트 입력값으로 신뢰하지 않고 인증 세션에서 가져온다.
- 시스템 프롬프트와 Tool 결과를 로그에서 구분한다.
- LLM 출력은 구조화된 DTO로 받은 후 검증한다.

RAG는 첫 단계의 필수 요소가 아니다. 도서 검색과 추천은 Tool Calling이 더 직접적이다. 리뷰 요약 근거 문장을 검색해야 할 때 pgvector 기반 RAG를 추가한다.

15. 배치와 모델 운영

15.1 정기 작업

작업	권장 주기
신규 도서 메타데이터 동기화	하루 1회
허용된 리뷰 증분 수집	하루 1회
신규 리뷰 감성 추론	수집 직후
도서별 감성 통계 재집계	하루 1회
트렌드 점수 갱신	1시간 또는 하루 1회
사용자 추천 재생성	하루 1회 + 주요 행동 직후
모델 재학습	데이터 증가량 기준 또는 수동 실행

15.2 작업 상태

`crawl_jobs`에 다음을 기록한다.

- job type
- started_at, finished_at

- status
- processed_count, success_count, failure_count
- source
- error summary
- retry count

작업은 재실행해도 중복 데이터가 생기지 않는 idempotent 구조로 만든다.

15.3 모델 버전

예시:

TEXT

```
sentiment-bilstm-0.2.0
sentiment-transformer-0.3.0
book-embedding-0.1.0
hybrid-recommender-0.3.0
```

모델 파일만 저장하지 말고 다음도 함께 기록한다.

- 학습 데이터 버전
- 전처리 코드 버전
- 하이퍼파라미터
- 평가 지표
- Git commit SHA
- 생성 일시
- 운영 적용 여부

16. 보안과 개인정보

- 비밀번호는 BCrypt 또는 Argon2로 해시한다.
- Access Token은 짧은 만료 시간을 사용한다.
- Refresh Token은 서버에서 폐기 가능하게 저장한다.
- 입력값 검증과 전역 예외 응답 형식을 통일한다.
- 로그인·챗봇·검색 API에 요청 제한을 적용한다.
- API 키와 DB 비밀번호는 저장소에 커밋하지 않는다.
- 운영 로그에 비밀번호, 토큰, 이메일 전문, LLM API 키를 남기지 않는다.
- 다른 사용자의 찜·추천·대화 기록을 조회할 수 없도록 소유권 검증을 한다.
- 개인화 이용 동의와 행동 기록 삭제 기능을 제공한다.
- 관리자 API는 별도 권한으로 제한한다.
- Python 내부 API는 외부 인터넷에 직접 노출하지 않는다.

LLM 제공자에게 전달되는 데이터에는 이메일, 토큰, 내부 사용자 ID 등 불필요한 개인정보를 포함하지 않는다.

17. 비기능 요구사항

성능 목표

- 일반 조회 API p95 500ms 이하
- 저장된 추천 결과 조회 p95 700ms 이하
- 검색 결과 첫 페이지 1초 이하
- 챗봇은 스트리밍을 사용하고 첫 응답 목표를 별도로 측정
- 1만 권, 리뷰 10만 건 규모에서 검색·집계 검증

위 값은 초기 목표이며 로컬, 테스트, 운영 환경을 구분해 측정한다.

안정성

- Python AI 서비스가 중단되어도 저장된 추천과 감성 결과는 조회 가능
- 외부 API 실패 시 지수 백오프와 최대 재시도 횟수 적용
- 수집 작업의 일부 실패가 전체 트랜잭션을 무효화하지 않도록 배치 단위 처리
- `/health`와 `/ready` 상태 확인 제공

데이터 품질

- ISBN 기준 중복 제거
- 원문과 정규화 문장을 분리
- 수집 출처와 시각 기록
- 분석 결과에 모델 버전 연결
- 결측치와 비정상 별점 검증

18. 테스트 전략

Java

- Service 단위 테스트
- Controller 슬라이스 테스트
- Spring Security 인가 테스트
- Testcontainers를 이용한 PostgreSQL 통합 테스트
- Flyway 마이그레이션 테스트
- 챗봇 Tool의 권한·빈 결과·오류 처리 테스트

Python

- 텍스트 정제 함수 단위 테스트
- 수집 파서의 고정 HTML fixture 테스트

- FastAPI 요청·응답 계약 테스트
- 모델 입력 shape와 출력 확률 검증
- 동일 모델·입력에 대한 재현성 검사
- 데이터 누수 검사

E2E

1. 회원가입
2. 취향 선택
3. 책 검색
4. 상세 조회
5. 찜
6. 추천 목록 변경 확인
7. 챗봇 추천

성능

- k6 또는 Gatling으로 검색·상세·추천 API 부하 테스트
- Python 배치 처리량 측정
- LLM 호출 횟수, 응답 시간, 실패율 측정

19. 배포 구조

포트폴리오용 권장안

DIAGRAM DEFINITION

flowchart TB

```

INTERNET["Internet"]
PROXY["Caddy/Nginx<br/>HTTPS"]
FRONT["React Static"]
BACK["Spring Boot Container"]
PY["Python AI Container"]
PG["PostgreSQL + pgvector"]

```

```

INTERNET --> PROXY
PROXY --> FRONT
PROXY --> /api| BACK
BACK --> PY
BACK --> PG
PY --> PG

```

처음에는 한 대의 클라우드 VM에 Docker Compose로 배포해도 충분하다. 중요한 것은 다음을 증명하는 것이다.

- 로컬과 운영 설정 분리
- 환경변수와 Secret 관리
- HTTPS

- 데이터 볼륨과 백업
- 자동 테스트
- 이미지 빌드
- 무중단까지는 아니더라도 재현 가능한 배포
- 장애 로그 확인 방법

CI/CD

Pull Request:

1. Java 테스트
2. Python 테스트
3. 프론트 테스트
4. 린트
5. Docker 이미지 빌드 검증

main 병합:

1. 버전 태그 또는 commit SHA로 이미지 생성
2. Container Registry push
3. 서버 배포
4. DB migration
5. health check
6. 실패 시 이전 이미지로 롤백

20. 저장소 구조

처음에는 모노레포가 관리하기 쉽다.

TEXT

```
bookpick-ai/
├── backend/
│   ├── src/main/java/
│   ├── src/test/java/
│   ├── build.gradle
│   └── Dockerfile
├── ai-service/
│   ├── app/
│   ├── pipelines/
│   ├── training/
│   ├── tests/
│   ├── pyproject.toml
│   └── Dockerfile
├── frontend/
│   ├── src/
│   ├── tests/
│   ├── package.json
│   └── Dockerfile
```

```

├─ infra/
│   ├── compose.yaml
│   ├── proxy/
│   └── monitoring/
├─ docs/
│   ├── architecture.md
│   ├── erd.md
│   ├── api.md
│   ├── model-card.md
│   └── adr/
├─ .github/workflows/
├─ .env.example
└─ README.md

```

21. 개발 일정 예시: 12주

주차	목표	결과물
1	요구사항-데이터 소스 검증	명세, 와이어프레임, 데이터 이용 조건
2	저장소-DB-Docker 기반 구성	모노레포, Compose, ERD, Flyway
3	회원-인증	인증 API와 테스트
4	도서 적재-검색-상세	수집 파이프라인, 검색 화면
5	찜-좋아요-행동 로그	사용자 기능과 이벤트 데이터
6	감성 분석 베이스라인	데이터셋, TF-IDF 모델, 평가 리포트
7	BiLSTM 학습	RNN 모델, 실험 비교
8	Transformer 비교-서비스 연동	최종 모델 선정, FastAPI
9	콘텐츠 기반 추천	개인 추천과 추천 이유
10	트렌드-하이브리드 추천	랭킹과 다양성
11	Spring AI 챗봇	Tool Calling, 스트리밍 UI
12	배포-모니터링-문서화	운영 URL, 시연 영상, README

시간이 부족하면 8주차 Transformer 비교와 10주차 협업 필터링을 축소한다. 인증-검색-감성 분석-콘텐츠 추천-챗봇-배포의 연결을 먼저 완성한다.

22. 포트폴리오 문서에 반드시 넣을 자료

README 첫 화면

- 서비스 한 줄 소개
- 배포 URL과 테스트 계정
- 1분 이내 시연 GIF 또는 영상

- 아키텍처
- 핵심 기능
- 기술 선택 이유
- 로컬 실행 방법

AI 모델 문서

- 문제 정의
- 데이터 출처와 사용 조건
- 라벨 분포
- 전처리
- 데이터 분리 방법
- TF-IDF, BiLSTM, Transformer 비교표
- 혼동 행렬
- 실패 사례
- 최종 모델 선택 이유
- 모델의 한계

추천 시스템 문서

- 이벤트와 가중치
- 후보 생성
- 랭킹
- 콜드 스타트
- 다양성 처리
- 오프라인 지표
- 사용자 테스트 결과

백엔드 문서

- 트랜잭션 경계
- 인증 구조
- 장애 대응
- 인덱스 전후 쿼리 성능
- Python 서비스 장애 시 fallback

기술 의사결정 기록 예시

- ADR-001: 마이크로서비스 대신 모듈형 모노리스 + Python 서비스 선택
- ADR-002: Elasticsearch 대신 PostgreSQL 검색으로 시작
- ADR-003: RNN을 최종 모델이 아닌 비교 모델로 사용
- ADR-004: 챗봇 검색에 RAG보다 Tool Calling 우선

- ADR-005: 크롤링보다 정식 Open API 우선

23. 면접에서 설명할 핵심 이야기

나쁜 설명:

Java와 Python을 사용했고 AI와 챗봇도 넣었습니다.

좋은 설명:

Spring Boot가 인증과 도메인 로직을 담당하고 Python 서비스가 NLP와 추천 추론을 담당하도록 역할을 분리했습니다. 리뷰 감성 분석은 TF-IDF, BiLSTM, 한국어 Transformer를 동일한 데이터 분할로 비교했고 Macro F1과 추론 비용을 기준으로 운영 모델을 선택했습니다. 추천은 콜드 스타트 문제 때문에 콘텐츠 기반으로 시작한 뒤 행동 로그와 시간 감쇠 트렌드 점수를 결합했습니다. 챗봇은 환각을 줄이기 위해 Spring AI Tool Calling으로 실제 도서 검색과 추천 API만 호출하도록 구현했습니다.

면접에서 중요한 것은 "무엇을 사용했다"보다 "어떤 문제가 있었고 왜 이렇게 결정했으며 결과가 어떻게 달라졌는지"다.

24. 완료 기준

다음 조건을 만족하면 1차 프로젝트를 완료한 것으로 본다.

- ☐ 공개된 운영 URL에서 서비스 사용 가능
- ☐ 테스트 계정으로 검색·찜·추천·챗봇 시연 가능
- ☐ 데이터 출처와 이용 조건 문서화
- ☐ 감성 모델 2개 이상 비교
- ☐ 추천 이유 표시
- ☐ 신규 사용자도 추천 가능
- ☐ Python 서비스 장애 시 기본 조회 가능
- ☐ Spring Boot-FastAPI OpenAPI 계약과 통합 테스트 통과
- ☐ FastAPI 응답에 모델 버전과 추론 시간 포함
- ☐ 핵심 API 자동 테스트
- ☐ Docker로 재현 가능
- ☐ CI/CD 성공
- ☐ ERD, API, 아키텍처, 모델 카드 작성
- ☐ 성능 또는 개선 수치 최소 2개 제시
- ☐ 알려진 한계와 향후 개선점 작성

25. 지금 결정해야 할 사항

개발 전에 아래 항목을 작은 실험으로 확인한다.

1. 합법적으로 사용할 수 있는 리뷰 원문을 충분히 확보할 수 있는가?
2. 국내 도서 메타데이터와 표지 이미지의 출처를 어디로 정할 것인가?
3. LLM은 외부 API와 로컬 모델 중 무엇을 사용할 것인가?
4. React에서 MVP로 구현할 화면과 2차로 미룰 화면은 무엇인가?
5. 총 개발 가능 기간과 주당 투입 시간은 얼마인가?
6. 지원하려는 직무는 Java 백엔드 중심인가, AI 서비스 백엔드 중심인가?

가장 먼저 해야 할 기술 실험은 다음 세 가지다.

- 국립중앙도서관 API로 도서 100권을 정상 수집해 DB에 넣기
- 허용된 리뷰 데이터 1,000건 이상을 확보해 감성 베이스라인 만들기
- Spring Boot에서 Python FastAPI를 호출하는 최소 통합 예제 만들기

이 세 가지가 성공하면 전체 프로젝트의 핵심 위험이 크게 줄어든다.

26. 참고한 공식 자료

- [Spring AI Reference](#)
- [Spring AI Getting Started](#)
- [Spring AI Tool Calling](#)
- [Spring AI RAG](#)
- [국립중앙도서관 ISBN 서지정보 Open API](#)
- [네이버 검색 책 API 서비스 종료 공지](#)
- [Open Library APIs](#)